

Step 1: Create a YAML File

Create a new file named:

```
ubuntu@ip-172-31-2-108:~$ touch pending-pod.yaml
ubuntu@ip-172-31-2-108:~$ nano pending-pod.yaml
ubuntu@ip-172-31-2-108:~$ kubectl apply -f pending-pod.yaml
pod/pending-demo created
ubuntu@ip-172-31-2-108:~$ cat pending-pod.yaml
apiVersion: v1
kind: Pod
metadata:
  name: pending-demo
spec:
  containers:
  - name: nginx
    image: nginx
    resources:
      requests:
        memory: "20Gi"
        cpu: "8"
ubuntu@ip-172-31-2-108:~$
```

Step 2: Deploy the Pod

Run:

```
kubectl apply -f pending-pod.yaml
```

Expected Output:

```
pod/pending-demo created
```

Step 3: Check the Pod Status

Run:

```
kubectl get pods
```

Expected Output:

NAME	READY	STATUS	RESTARTS	AGE
pending-demo	0/1	Pending	0	10s

The Pod is created but cannot be scheduled.

```

cpu: 8
ubuntu@ip-172-31-2-108:~$ kubectl get pods
NAME          READY   STATUS             RESTARTS   AGE
crashloop-demo 0/1     ImagePullBackOff   0           7m13s
pending-demo   0/1     Pending            0           51s
ubuntu@ip-172-31-2-108:~$

```

Step 4: Describe the Pod

Run:

```
kubectl describe pod pending-demo
```

Scroll to the **Events** section.

Expected Output:

Events:

Warning FailedScheduling

0/1 nodes are available:

Insufficient cpu

Insufficient memory

This tells you that Kubernetes cannot find a node with enough resources.

```

ubuntu@ip-172-31-2-108:~$ kubectl describe pod pending-demo
Name:          pending-demo
Namespace:     default
Priority:       0
Service Account: default
Node:          <none>
Labels:        <none>
Annotations:   <none>
Status:        Pending
IP:            <none>
IPs:           <none>
Containers:
  nginx:
    Image:      nginx
    Port:       <none>
    Host Port:  <none>
    Requests:
      cpu:        8
      memory:     20Gi
    Environment: <none>
    Mounts:
      /var/run/secrets/kubernetes.io/serviceaccount from kube-api-access-vlzd1 (ro)
Conditions:
  Type             Status
  PodScheduled     False
Volumes:
  kube-api-access-vlzd1:
    Type:          Projected (a volume that contains injected data from multiple sources)
    TokenExpirationSeconds: 3607
    ConfigMapName:  kube-root-ca.crt

```

```
IP:
IPs: <none>
Containers:
  nginx:
    Image: nginx
    Port: <none>
    Host Port: <none>
    Requests:
      cpu: 8
      memory: 20Gi
    Environment: <none>
    Mounts:
      /var/run/secrets/kubernetes.io/serviceaccount from kube-api-access-vlzd1 (ro)
Conditions:
  Type          Status
  PodScheduled  False
Volumes:
  kube-api-access-vlzd1:
    Type: Projected (a volume that contains injected data from multiple sources)
    TokenExpirationSeconds: 3607
    ConfigMapName: kube-root-ca.crt
    Optional: false
    DownwardAPI: true
QoS Class: Burstable
Node-Selectors: <none>
Tolerations:
  node.kubernetes.io/not-ready:NoExecute op=Exists for 300s
  node.kubernetes.io/unreachable:NoExecute op=Exists for 300s
Events:
  Type      Reason      Age   From      Message
  ----      -
  Warning   FailedScheduling  79s   default-scheduler  0/1 nodes are available: 1 Insufficient cpu, 1 Insufficient memory. no new claims to deallocate, pr
emption: 0/1 nodes are available: 1 Preemption is not helpful for scheduling.
ubuntu@ip-172-31-2-108:~$
```

Step 5: Check Node Resources

Run:

```
kubectl describe node minikube
```

Look for:

Capacity

Allocatable

Allocated resources

This shows:

- Total CPU
- Total Memory
- Available CPU
- Available Memory

Compare these values with your Pod's requested resources.

Step 6: Verify Cluster Nodes

Run:

```
kubectl get nodes
```

Expected Output:

NAME	STATUS	ROLES	AGE
minikube	Ready	control-plane	

Your cluster has only one node, which does not have enough CPU or memory to satisfy the request.

Step 7: Check Pod Events

Run:

```
kubectl get events --sort-by=.metadata.creationTimestamp
```

Expected Output:

FailedScheduling

0/1 nodes are available:

Insufficient cpu

Insufficient memory

This provides a timeline of scheduling events.

Step 8: Fix the Issue

Edit the YAML file:

```
vi pending-pod.yaml
```

Replace the resources section with:

```
resources:
```

```
  requests:
```

```
    cpu: "100m"
```

```
    memory: "128Mi"
```

The updated YAML becomes:

```
apiVersion: v1
kind: Pod
metadata:
  name: pending-demo
spec:
  containers:
  - name: nginx
    image: nginx
  resources:
    requests:
      cpu: "100m"
      memory: "128Mi"
```

Save and exit.

Step 9: Delete the Old Pod

```
kubectl delete pod pending-demo
```

Expected Output:

```
pod "pending-demo" deleted
```

Step 10: Deploy the Updated Pod

```
kubectl apply -f pending-pod.yaml
```

Expected Output:

```
pod/pending-demo created
```

```
ubuntu@ip-172-31-2-108:~$ cat pending-pod.yaml
apiVersion: v1
kind: Pod
metadata:
  name: pending-demo
spec:
  containers:
  - name: nginx
    image: nginx
    resources:
      requests:
        memory: "128Mi"
        cpu: "100m"
ubuntu@ip-172-31-2-108:~$
```

Step 11: Verify the Pod

Run:

```
kubectl get pods
```

Expected Output:

NAME	READY	STATUS	RESTARTS	AGE
pending-demo	1/1	Running	0	15s

The Pod is now successfully scheduled and running.

Commands Used

```
kubectl apply -f pending-pod.yaml
```

```
kubectl get pods
```

```
kubectl describe pod pending-demo
```

```
kubectl describe node minikube
```

```
kubectl get nodes
```

```
kubectl get events --sort-by=.metadata.creationTimestamp
```

```
kubectl delete pod pending-demo
```

```
kubectl apply -f pending-pod.yaml
```

```
kubectrl get pods
```

Root Cause

The Pod requested:

- **CPU:** 8 cores
- **Memory:** 20 GiB

However, the Minikube node did not have sufficient available resources. Since Kubernetes could not find a node that met these requirements, the scheduler left the Pod in the **Pending** state.

Resolution

Reduce the CPU and memory requests to values that fit within the node's available capacity, then redeploy the Pod.
